Layer-Freezing Sweep Experiment

This experiment is part of a broader research effort applying federated learning to disaster prediction (wildfires, floods, earthquakes). Federated training in this setting is combined with differential privacy (DP-SGD via Opacus, noise multiplier = 1.1) to protect data from participating clients. Since DP-SGD injects noise proportional to the number of trainable parameters, the number of unfrozen layers directly affects both model utility and the privacy budget (epsilon).

Before tuning noise multipliers and estimating epsilon budgets, we first needed to establish how many pretrained layers should remain trainable during federated fine-tuning. This sweep isolates that variable using private federated training, so that the effect of layer freezing can be measured cleanly.

Method

Framework: Flower for federated orchestration.

Three pretrained architectures were swept, each across a range of "trainable layer" configurations:

- DistilBERT (text)
- MobileBERT (text)
- MobileNetV3 (image)

For each architecture, models were trained with an increasing number of unfrozen layers, denoted by an integer id:

- -1: only the final classifier layer is trainable; all other layers (including the first classifier layer) are locked.
- 0: the full classifier head is trainable; all backbone layers are locked.
- 1, 2, 3, ...: the classifier head plus the last N layers of the pretrained backbone are trainable.

Datasets used were standard defaults rather than disaster-specific data, to validate the sweep methodology before applying it to domain-specific datasets: Emotion for the text models (DistilBERT, MobileBERT) and CIFAR-10 for the image model (MobileNetV3). Accuracy and loss were logged per federated round for each

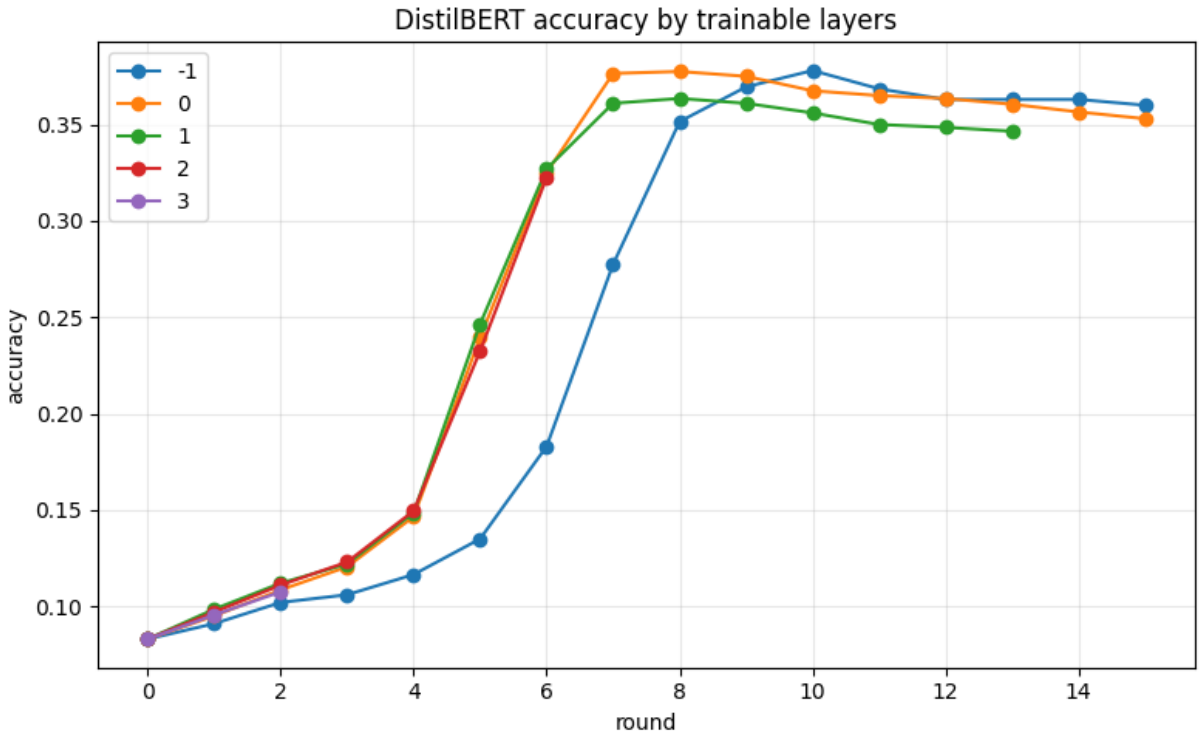
In [1]:

```
import pandas
import matplotlib.pyplot as plt

def flwr_accuracy(folder, key, title, exclude_ids = {}):
    df = pandas.read_csv(folder + "/flwr.csv")
    df = df[~df["id"].isin(exclude_ids)]
    plt.figure(figsize=(8,5))
    for id_val, g in df.groupby("id"):
        g = g.sort_values("round")
        plt.plot(g["round"], g[key], marker="o", label=str(id_val))

    plt.xlabel("round")
    plt.ylabel(key)
    plt.title(title)
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()
```

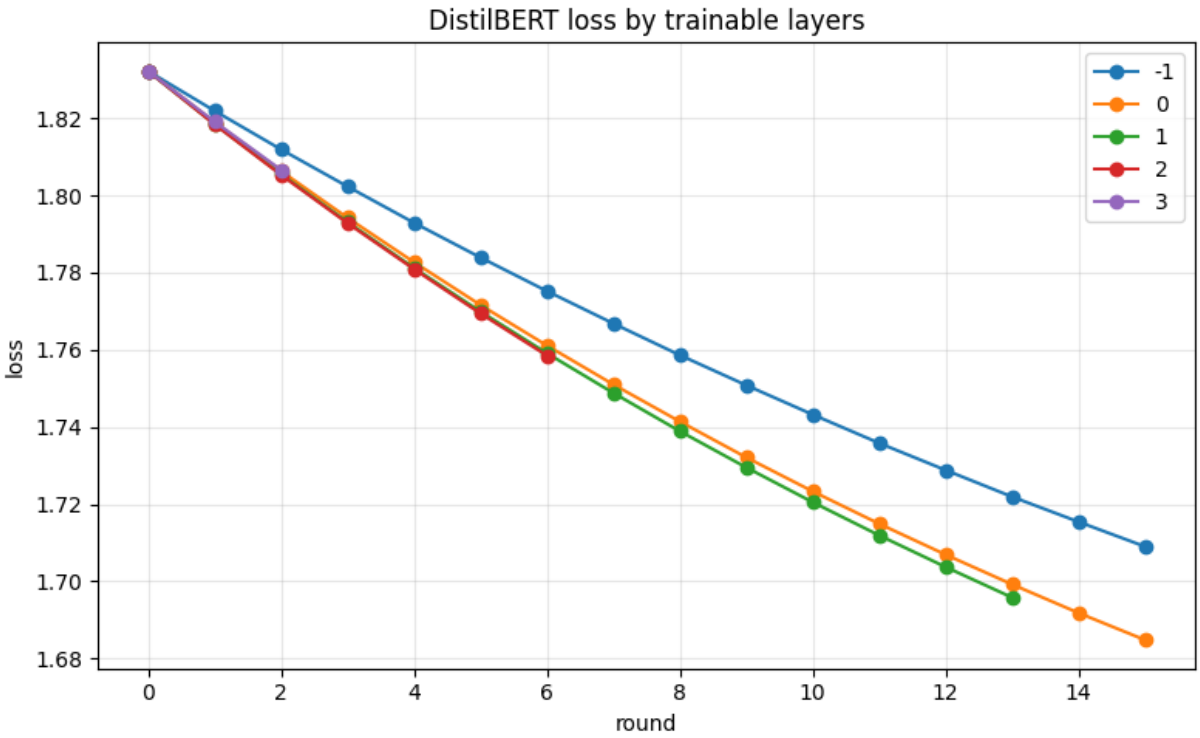
DistilBERT, 15 rounds

noise-multiplier: 1.1, max-grad-norm: 1.0, learning-rate: 0.00002, batch-size: 32

In [2]:
```
flwr_accuracy("distil_layers", "accuracy", "DistilBERT accuracy by trainable
```



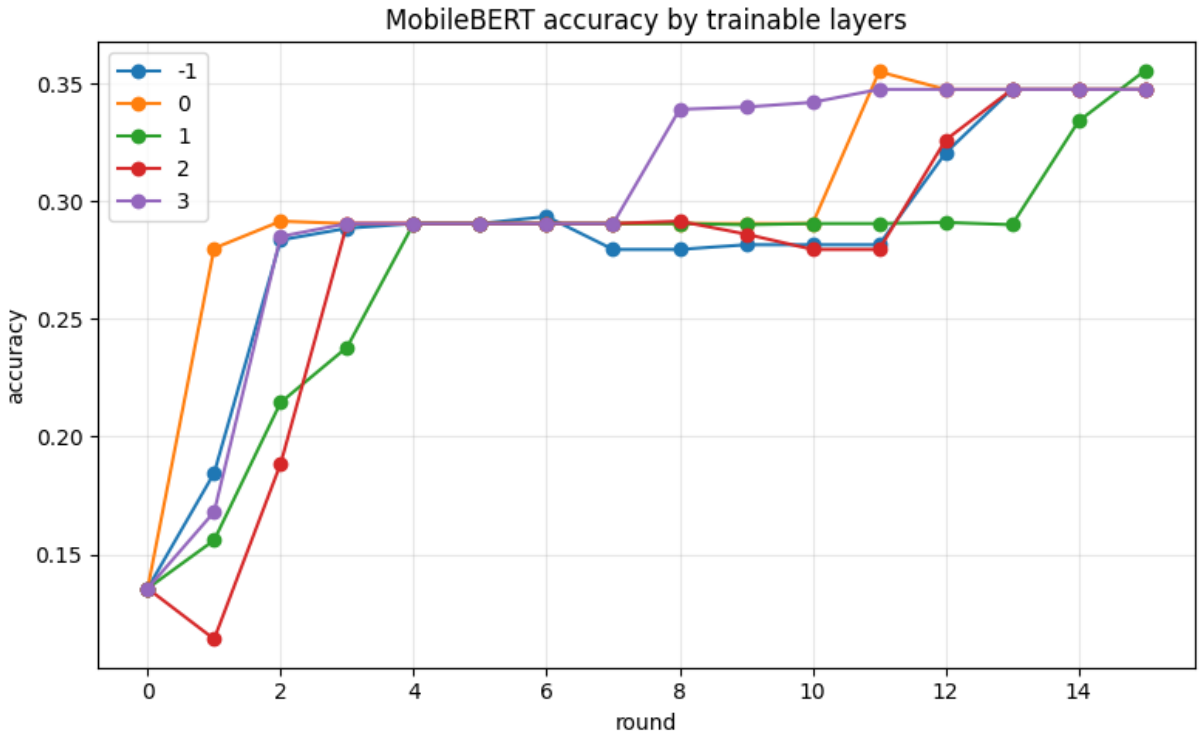
In [3]:
```
flwr_accuracy("distil_layers", "loss", "DistilBERT loss by trainable layers"
```



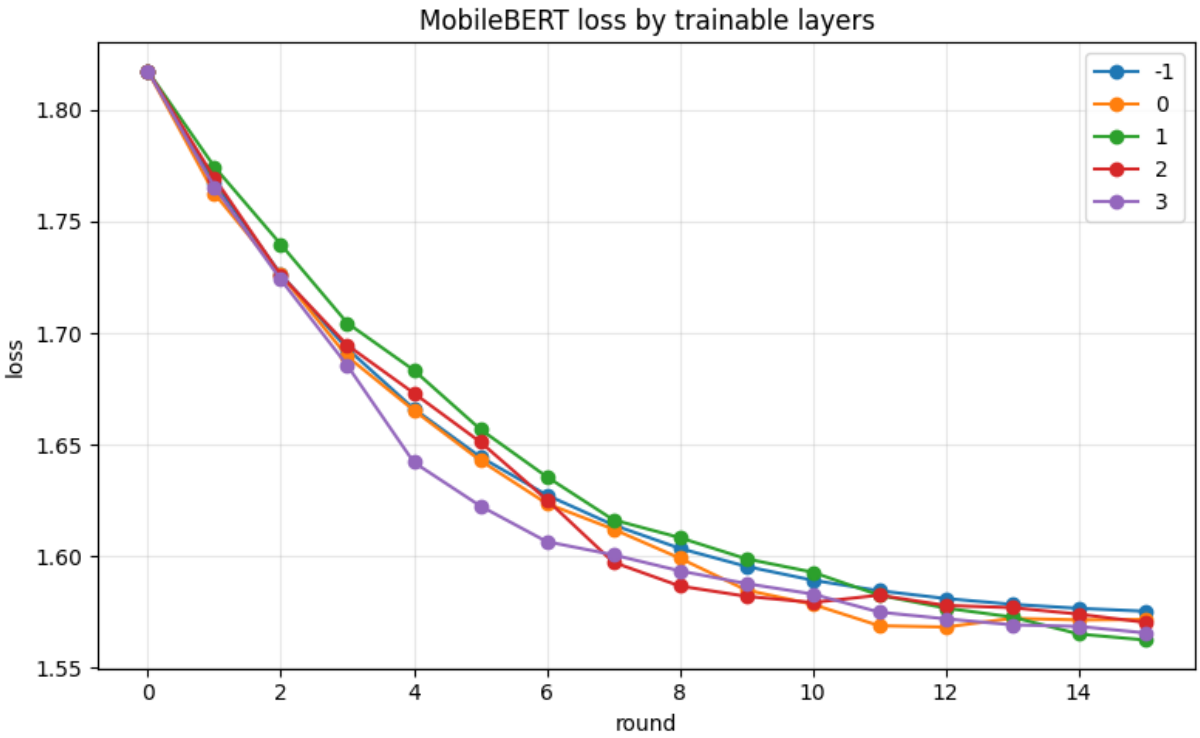
MobileBERT, 15 rounds

noise-multiplier: 1.1, max-grad-norm: 1.0, learning-rate: 0.00002, batch-size: 32

In [4]:
```
flwr_accuracy("bert_layers", "accuracy", "MobileBERT accuracy by trainable l
```



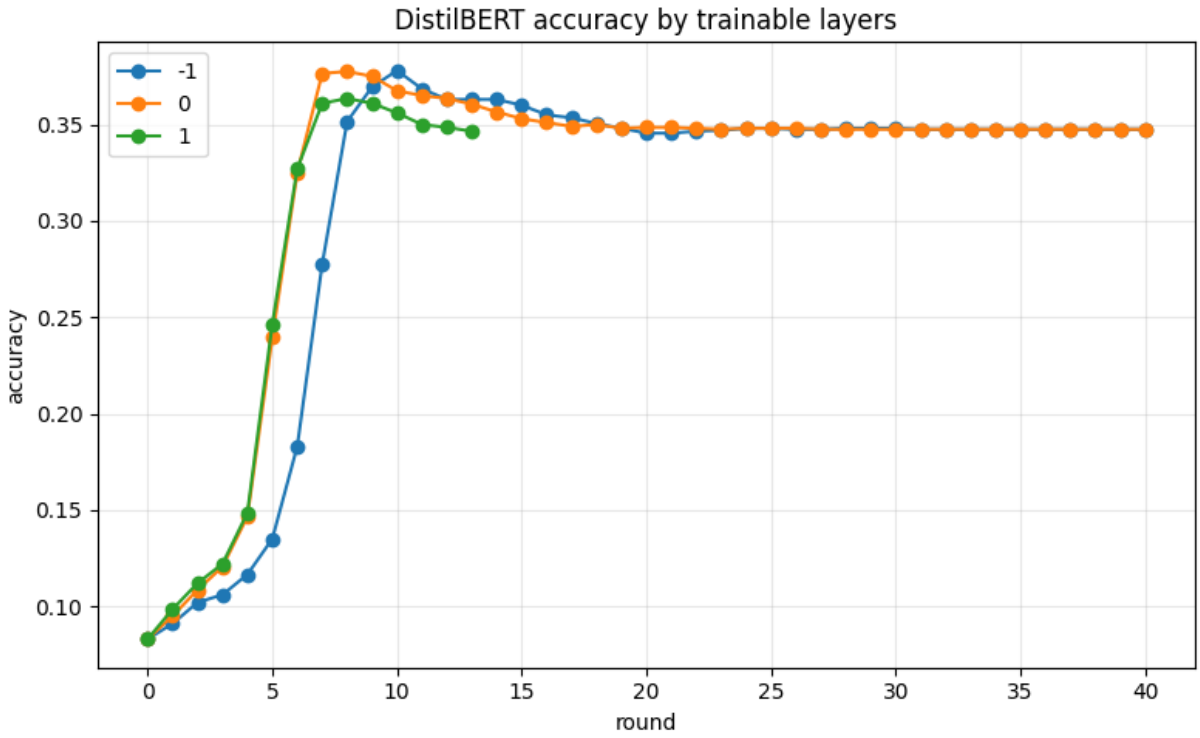
In [5]:
```
flwr_accuracy("bert_layers", "loss", "MobileBERT loss by trainable layers",
```



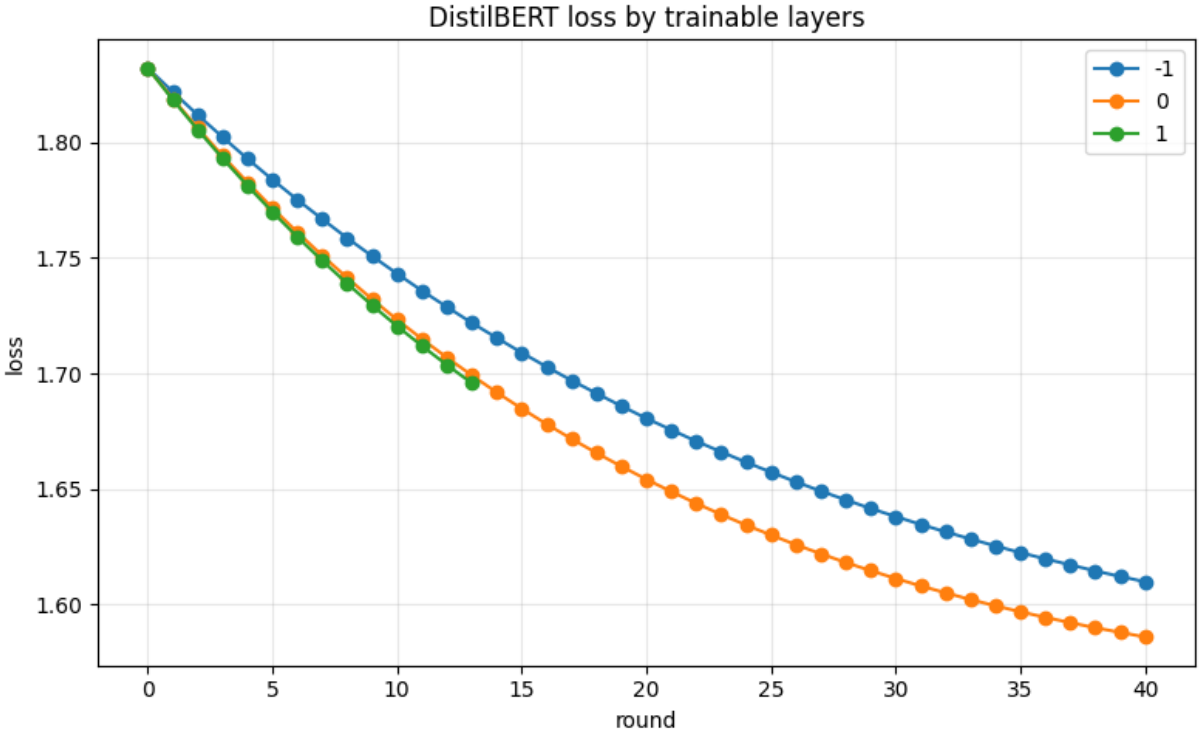
DistilBERT, 40 rounds

noise-multiplier: 1.1, max-grad-norm: 1.0, learning-rate: 0.00002, batch-size: 32

In [6]:
```
flwr_accuracy("distil_layers_40", "accuracy", "DistilBERT accuracy by traina
```



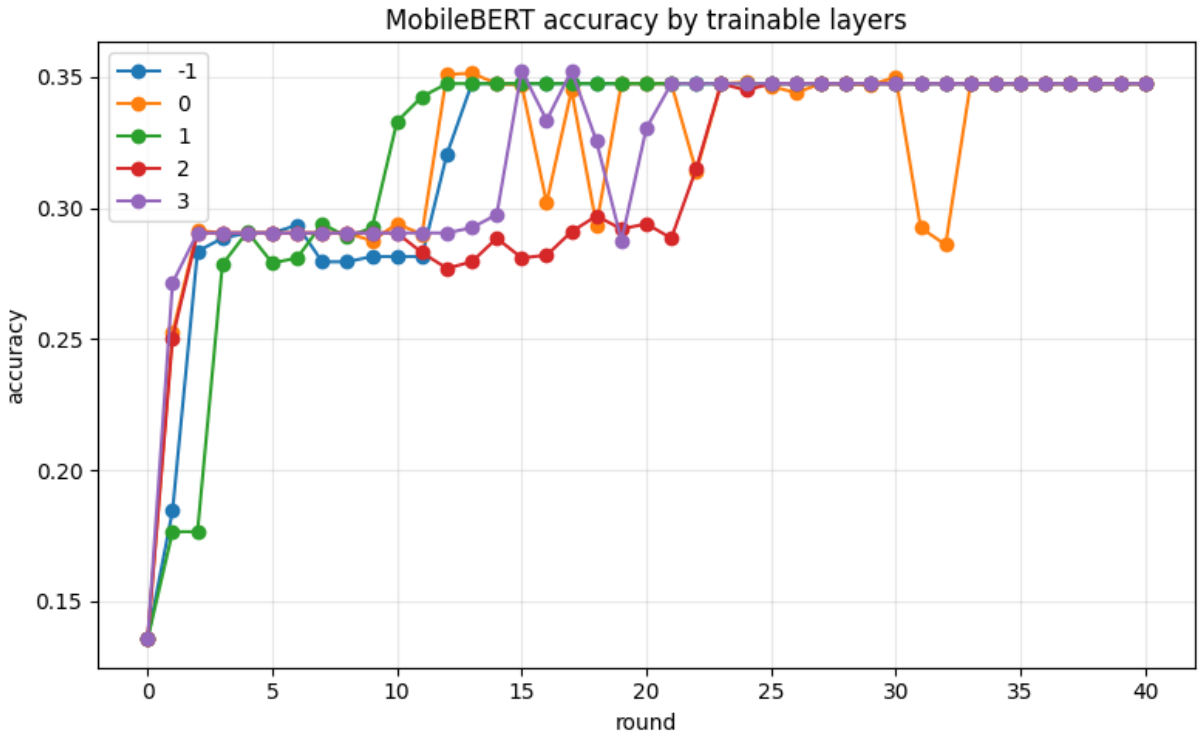
In [7]:
```
flwr_accuracy("distil_layers_40", "loss", "DistilBERT loss by trainable laye
```



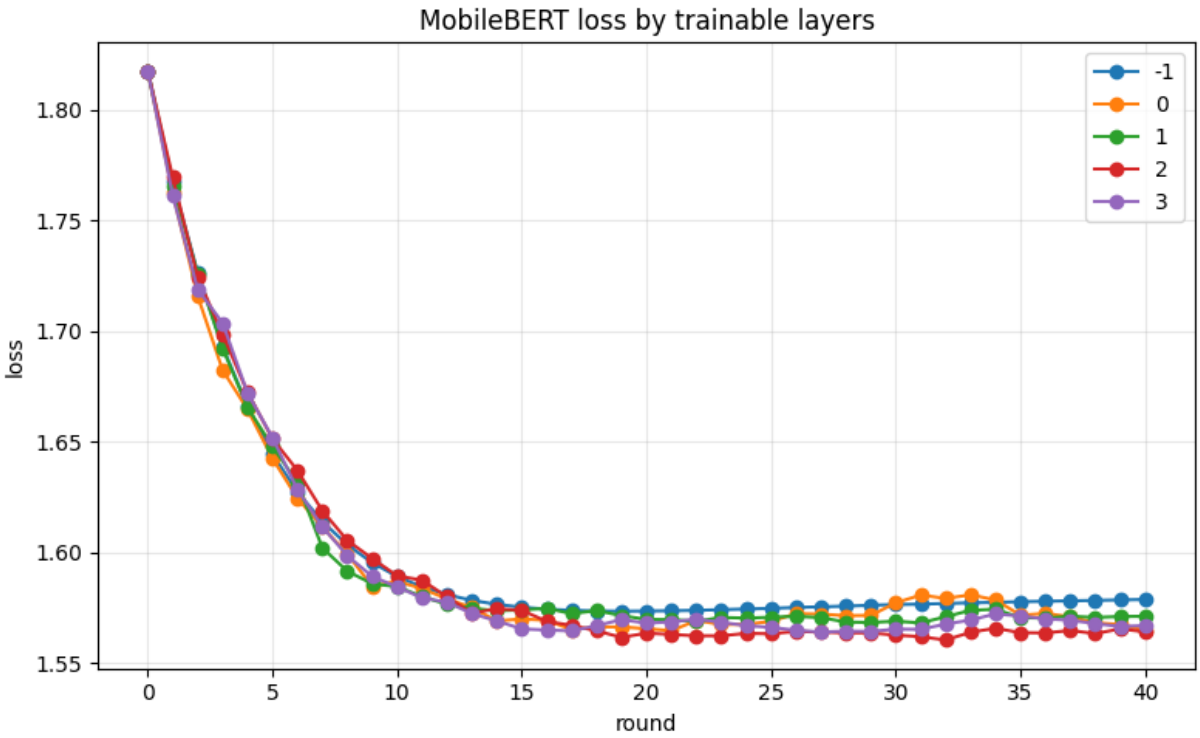
MobileBERT, 40 rounds

noise-multiplier: 1.1, max-grad-norm: 1.0, learning-rate: 0.00002, batch-size: 32

In [8]:
```
flwr_accuracy("bert_layers_40", "accuracy", "MobileBERT accuracy by trainabl
```



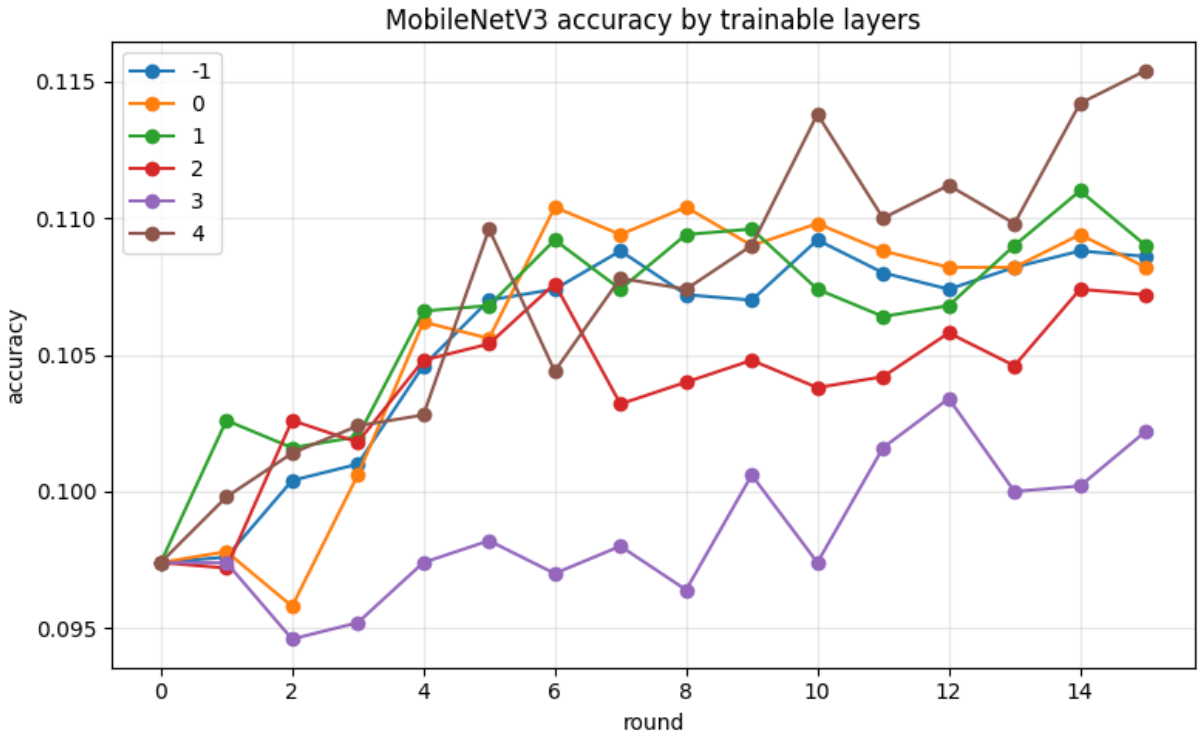
In [9]:
```
flwr_accuracy("bert_layers_40", "loss", "MobileBERT loss by trainable layers
```



MobileNetV3

noise-multiplier: 1.1, max-grad-norm: 1.0, learning-rate: 0.001, batch-size: 32

In [10]:
```
flwr_accuracy("mn_layers", "accuracy", "MobileNetV3 accuracy by trainable la
```



In [11]:
```
flwr_accuracy("mn_layers", "loss", "MobileNetV3 loss by trainable layers")
```